

Computer Vision Internship Assignment: Multi-View Person Re-Identification

1. Context

Consider a retail store, a campus, or a stadium covered by several cameras with no overlap in their fields of view. A person walks past camera A and, a minute later, appears in front of camera C. The system needs to recognise that these two sightings are the same individual, using full-body appearance rather than face recognition. This is the multi-camera (or multi-view) person re-identification problem, usually shortened to ReID. It sits behind retail analytics, smart-city, and sports vision systems, and it is close to the kind of work we do at Big Vision.

Your task is to build and demonstrate an end-to-end pipeline for this problem. The pipeline should detect people, produce an embedding for each detection, match the same person across camera views, visualise the results, and present them in a small dashboard.

We are more interested in how you approach the problem than in a state-of-the-art leaderboard score. We want to see how you scope the task, choose your tools, train your models, evaluate your results carefully, and explain your reasoning.

2. What we're evaluating

This assignment helps us understand your ability across four areas. Each stage of the task exercises one or more of them.

1. **Computer vision understanding.** Detection versus ReID, embeddings and metric learning, the multi-camera matching problem, and how ReID is evaluated (query and gallery sets, the cross-camera protocol, mAP and CMC).
2. **Model training.** Training and fine-tuning models properly: loss design, samplers, augmentation, avoiding data leakage, reading training curves, and running a controlled experiment.
3. **OpenCV and classical CV.** Handling video and image I/O, cropping, drawing, and building clear visual output.
4. **Software engineering and code quality.** Whether your pipeline is clean, modular, reproducible, and readable.

We also pay close attention to documentation and communication. Your notebook should read as a clear account of the problem and how you solved it, not a wall of code.

One thing to keep in mind: we would rather see a smaller pipeline that runs end to end with clear analysis than a large one that only half works and leans on copied, unexplained code. A pipeline that works matters more than an ambitious one that does not, and clear reasoning matters more than an inflated number.

3. The task

The task has core stages, which are all required, and a stretch section, from which you should pick at least one item. The stretch section is deliberately open-ended and is where you can show more depth.

Stage 0: Choose and understand your dataset(s)

- Select a public dataset, or datasets, for multi-camera person ReID, and explain in writing why you chose it.
- Show that you understand the dataset by documenting the number of cameras and views, the number of identities, the train, query, and gallery split, and the standard evaluation protocol. Explain why ReID uses a query and gallery setup, why matching is done across cameras, and why same-camera same-identity matches are excluded when scoring.
- Licensing and ethics: check the licence, consent basis, and provenance of any dataset you use. Some widely cited ReID datasets have been withdrawn over privacy and consent concerns and should not be used, so we want to see that you check this and make a documented, responsible choice. A short paragraph on the ethical considerations of ReID as a technology is welcome.
- A practical note on data: ReID training sets are usually pre-cropped identity images, but the end-to-end demo in Stages 3 and 4 needs raw multi-camera frames or video. Explain how you handle this. For example, you might use one dataset for ReID training and a separate raw multi-camera source for the demo, such as a multi-camera tracking dataset or a short clip you record from a few angles. If raw multi-camera footage is hard to obtain, you may demonstrate cross-view matching using the dataset's own multi-camera gallery, as long as you are explicit about what you did.

Some starting points, none of which are mandatory as long as you justify your choice: Market-1501, which has 6 non-overlapping cameras and roughly 1,501 identities and is a clean, widely used benchmark; or MSMT17, which has 15 cameras and is larger and considerably harder, though its access is gated. For raw multi-camera video, options include multi-camera people-tracking datasets or a short multi-angle clip you record yourself.

Stage 1: Person-only detector

- Build a detector that finds people only. Start from a modern pretrained detector and adapt it to the single person class. You could fine-tune on a person-detection dataset, restrict or retrain the head, or take another approach you can justify.
- Filtering the person class from a COCO-pretrained model at inference is acceptable as a baseline, but we want to see some real training or fine-tuning so that we can judge your training skills. Describe what you did and why.
- Report detection metrics: mAP@0.5, mAP@0.5:0.95, and precision and recall. Report inference speed as FPS together with the hardware you ran on. Include a few qualitative detection images.
- Example detectors include YOLO11, YOLO26, RT-DETR, or another well-justified choice.

Stage 2: Re-identification model

- Train or fine-tune a ReID model that maps a person crop to an embedding vector, so that crops of the same identity are close together and crops of different identities are far apart across different cameras.
- We want to see real training understanding rather than a single black-box call. At a minimum, expose and explain your backbone choice, the loss (for example, ID cross-entropy combined with triplet loss and hard mining, label smoothing, a BNNeck, or a margin-based loss such as ArcFace), the sampler (identity-balanced, or PK, sampling), the augmentations, the optimiser and schedule, and how you prevent train and test identity leakage. You may use libraries such as torchreid, fast-reid, Ultralytics, or timm for scaffolding, but you should be able to explain every component and run at least one meaningful experiment.
- Report results on the standard protocol: mAP and CMC Rank-1, Rank-5, and Rank-10. Include a query-to-gallery ranked-retrieval visualisation, showing a query image with its top-k gallery matches and marking which are correct, and include a distance or similarity matrix.

Stage 3: Multi-view matching and visualisation with OpenCV

- Connect Stages 1 and 2 on raw multi-camera footage. For each frame from each camera, detect people, crop them, produce embeddings, and assign a consistent global ID to the same person seen across different camera views.
- Use OpenCV for the video and image I/O, the cropping, and the drawing. Draw bounding boxes labelled with the global ID, use a consistent colour per identity across all views, lay the camera views out side by side or in a grid, and show a gallery strip of the crops matched to a chosen identity.
- Produce a short annotated output, such as an image montage, a GIF, or a video, that clearly shows the same person recognised across views.

Stage 4: Dashboard

Build a small dashboard that shows the whole pipeline in one place. It should cover four things:

1. Detection results and metrics.
2. Re-identification results, including mAP, the CMC curve, and Rank-k numbers.
3. Cross-view matches, so you can pick a person and see all of their appearances across the different cameras.
4. Interactive exploration. For example, select a query and view its ranked matches, or adjust the matching threshold and watch how the matches change.

Use whatever you prefer, such as Streamlit, Gradio, Plotly Dash, or ipywidgets inside the notebook. If it cannot run inline in the notebook, include screenshots or a GIF along with the code needed to launch it.

Stretch: add complexity

Pick at least one of the following; more than one is welcome. Choose based on your interests and the time you have. One item done well, with careful analysis, is worth more than several done superficially.

- **Full multi-camera tracking.** Add a tracker such as ByteTrack, BoT-SORT, or SORT to produce tracklets within each camera, then link tracklets across cameras using ReID. Discuss how you handle ID switches and fragmentation. This is the fuller version of the problem.
- **Re-ranking.** Implement k-reciprocal re-ranking and report the gain in mAP and Rank-1 that it gives you.
- **Backbone or loss ablation.** Compare at least two configurations, for example a lightweight CNN embedding against a ViT or CLIP-based embedding, or triplet-only against ID plus triplet against ArcFace, and analyse the trade-offs in accuracy, model size, and speed.
- **Cross-domain generalisation.** Train on one dataset and evaluate on another without fine-tuning. Report the drop in performance and explain why ReID models tend to overfit to the specific cameras and appearance distribution of their training set. This is a well-known and important failure mode in practice.
- **Robustness.** Evaluate on harder cases such as occlusion and low resolution, and discuss the results.
- **Efficiency and deployment.** Measure end-to-end latency and throughput, and discuss what it would take to run this at the edge or scale it to many cameras.
- **Failure analysis.** Systematically characterise where matching breaks, for example under similar clothing, lighting changes, or pose changes, with concrete examples.

4. Resources and constraints

- Compute: use Kaggle notebooks, Google Colab (free or Pro), or any environment you have access to. Work within free-tier limits. A smaller experiment run cleanly on a subset is completely fine and often shows better judgement than an overly ambitious run that never finishes.
- Libraries and references: use any open-source libraries and pretrained weights, and consult tutorials and papers freely. Cite what you use, and make sure the work and the reasoning are your own. Copied, unexplained code counts against you.
- Reproducibility: set random seeds, pin key package versions, and make it possible for us to re-run, or at least clearly follow, your notebook from top to bottom.

5. Deliverable

Submit a single, well-documented Jupyter or IPython notebook (a .ipynb file) that presents the work from start to finish. It should interleave your reasoning, decisions, trade-offs, code, results, and analysis in markdown cells, so that a reader can go through it from top to bottom and understand both what you did and why.

Near the top, include a short section on your approach and key decisions. Throughout, explain your choices: why this dataset, why this detector, why this loss, what surprised you, and what you would do with more time.

Do not embed trained weights or datasets in the notebook. Host weights on Google Drive or Hugging Face and link to them, and refer to datasets by name or URL. Include the exact commands and configuration needed to reproduce your results.

A GitHub repository is optional but appreciated, as are the demo GIF or video and dashboard screenshots, embedded or linked in the notebook.

6. Tips

- Get a thin end-to-end version working first, even on a tiny subset, and then improve it. A complete pipeline is better than a perfect detector with no ReID attached.
- Report honest numbers. A clear account of what did not work is more valuable than inflated results.
- Watch the evaluation pitfalls. Do not let training identities leak into the test set, follow the standard query and gallery protocol, and exclude same-camera same-identity matches when computing ReID metrics.
- When something is ambiguous, state your assumption and continue. Reasonable, well-documented assumptions are exactly what we want to see.

7. Questions

If anything is unclear, feel free to ask. Reaching out with good questions is a positive signal, not a negative one.

Good luck.